

Отчёт по лабораторной работе №1

Дисциплина: Сетевые технологии

true

Содержание

Цель работы

Целью данной работы является изучение методов кодирования и модуляции сигналов с помощью высокоуровневого языка программирования Octave. Определение спектра и параметров сигнала. Демонстрация принципов модуляции сигнала на примере аналоговой амплитудной модуляции. Исследование свойства самосинхронизации сигнала.

Задание

1. Построить график функции

$$y = \sin(x) + \frac{1}{3}\sin(3x) + \frac{1}{5}\sin(5x)$$

на интервале $[-10; 10]$, используя Octave и функцию `plot`. График экспортировать в файлы формата `.eps`, `.png`.

2. Добавить график функции

$$y = \cos(x) + \frac{1}{3}\cos(3x) + \frac{1}{5}\cos(5x)$$

на интервале $-10; 10$. График экспортировать в файлы формата `.eps`, `.png`.

3. Разработать код m-файла, результатом выполнения которого являются графики меандра, реализованные с различным количеством гармоник.
4. Определить спектр двух отдельных сигналов и их суммы
5. Выполнить задание с другой частотой дискретизации. Пояснить, что будет, если взять частоту дискретизации меньше 80 Гц?
6. Продемонстрировать принципы модуляции сигнала на примере аналоговой амплитудной модуляции
7. По заданных битовых последовательностей требуется получить кодированные сигналы для нескольких кодов, проверить свойства самосинхронизуемости кодов, получить спектры.

Выполнение лабораторной работы

Построение графиков в Octave

Сначала установили в нашей ОС Octave: `sudo apt install octave` ([рис. @fig-001]), ([рис. @fig-002])

Далее запустили в нашей ОС Octave с оконным интерфейсом ([рис. @fig-003])

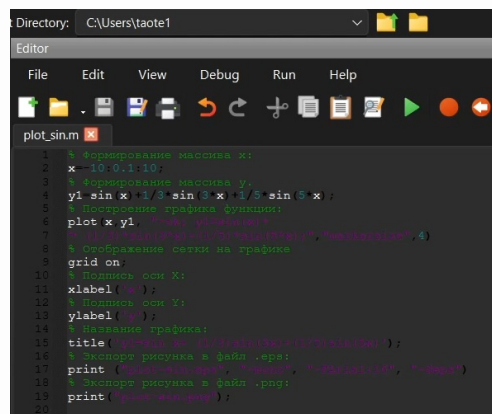
Далее перешли в окно редактора. Воспользовавшись комбинацией клавиш `ctrl + n` создали новый сценарий. Сохранили его в наш рабочий каталог с именем `plot_sin.m`. В окне редактора написали код по построению графика функции

$$y = \sin(x) + \frac{1}{3}\sin(3x) + \frac{1}{5}\sin(5x)$$

на интервале $[-10; 10]$:

```
% Формирование массива x:
x=-10:0.1:10;
% Формирование массива y.
y1=sin(x)+1/3*sin(3*x)+1/5*sin(5*x);
% Построение графика функции:
plot(x,y1, "-ok; y1=sin(x)+(1/3)*sin(3*x)+(1/5)*sin(5*x);", "markersize",4)
% Отображение сетки на графике
grid on;
% Подпись оси X:
xlabel('x');
% Подпись оси Y:
ylabel('y');
% Название графика:
title('y1=sin x+ (1/3)sin(3x)+(1/5)sin(5x)');
% Экспорт рисунка в файл .eps:
print ("plot-sin.eps", "-mono", "-FArial:16", "-deps")
% Экспорт рисунка в файл .png:
print("plot-sin.png");
```

После этого запустили наш сценарий, воспользовавшись клавишей f5. Открылось окно с построенным графиком и в нашем рабочем каталоге появились файлы с графиками в форматах .eps, .png ([рис. @fig-004]), ([рис. @fig-005])



Работа программы plot_sin.m

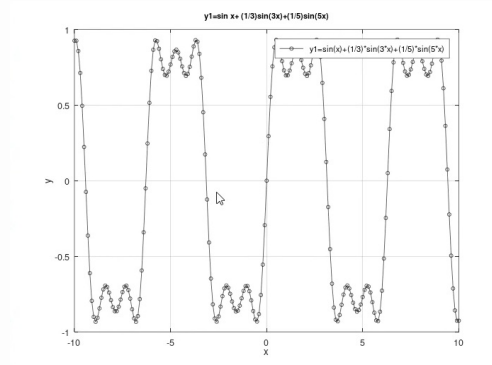


График функции y

Далее сохранили сценарий под другим названием *plot_sic_cos.m* и изменили код так чтобы на одном графике располагались отличающиеся по типу линий графики функций

$$y_1 = \sin(x) + \frac{1}{3}\sin(3x) + \frac{1}{5}\sin(5x)$$

и

$$y_2 = \cos(x) + \frac{1}{3}\cos(3x) + \frac{1}{5}\cos(5x)$$

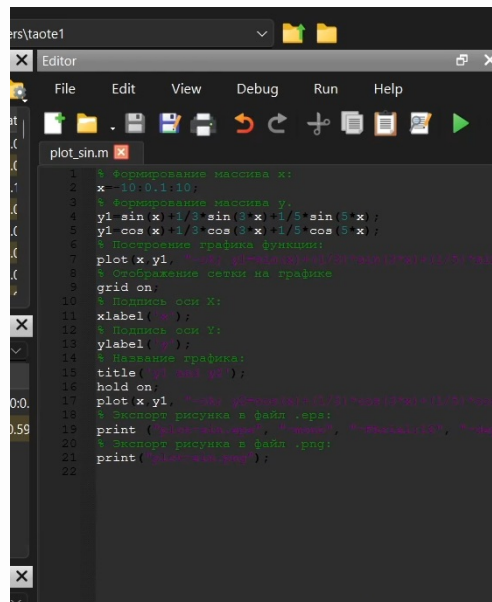
на интервале $[-10; 10]$:

```

% Формирование массива x:
x=-10:0.1:10;
% Формирование массива y.
y1=sin(x)+1/3*sin(3*x)+1/5*sin(5*x);
y2=cos(x)+1/3*cos(3*x)+1/5*cos(5*x);
% Построение графика функции:
plot(x,y1, "-ok; y1=sin(x)+(1/3)*sin(3*x)+(1/5)*sin(5*x);","markersize",4,
x,y2,
"-k; y2=cos(x)+(1/3)*cos(3*x)+(1/5)*cos(5*x);","markersize",4)
% Отображение сетки на графике
grid on;
% Подпись оси X:
xlabel('x');
% Подпись оси Y:
ylabel('y');
% Название графика:
title('y1=sin x+ (1/3)sin(3x)+(1/5)sin(5x); y2=cos x+ (1/3)cos(3x)+
(1/5)cos(5x)');
% Экспорт рисунка в файл .eps:
print ("plo_sin_cos.eps", "-mono", "-FArial:16", "-deps")
% Экспорт рисунка в файл .png:
print("plot_sin_cos.png");

```

И запустили сценарий ([рис. @fig-006]), ([рис. @fig-007])



Работа программы plot_sin_cos.m

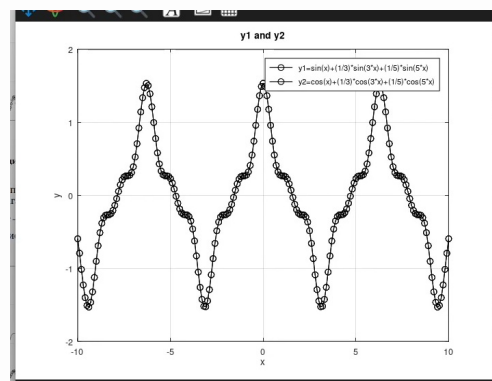


График функций y1 и y2

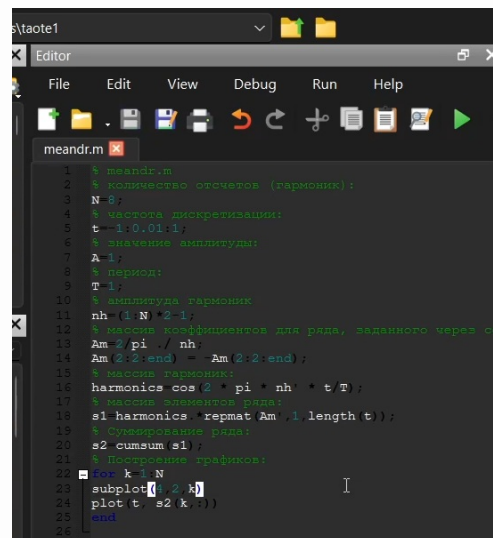
Разложение импульсного сигнала в частичный ряд

Фурье

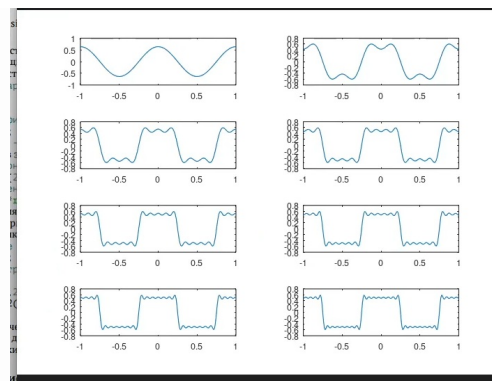
Далее создали новый сценарий и сохранили его в наш рабочий каталог с именем *meandr.m*. В нём реализовали код для реализации меандра через косинусы:

```
% meandr.m
% количество отсчетов (гармоник):
N=8;
% частота дискретизации:
t=-1:0.01:1;
% значение амплитуды:
A=1;
% период:
T=1;
% амплитуда гармоник
nh=(1:N)*2-1;
% массив коэффициентов для ряда, заданного через cos:
Am=2/pi ./ nh;
Am(2:2:end) = -Am(2:2:end);
% массив гармоник:
harmonics=cos(2 * pi * nh' * t/T);
% массив элементов ряда:
s1=harmonics.*repmat(Am',1,length(t));
% суммирование ряда:
s2=cumsum(s1);
% Построение графиков:
for k=1:N
    subplot(4,2,k)
    plot(t, s2(k,:))
end
```

Запустили сценарий и получили график меандра через косинусы ([рис. @fig-008]), ([рис. @fig-009])



Работа программы meandr.m (1)



Графики меандра, содержащего различное число гармоник (через косинусы)

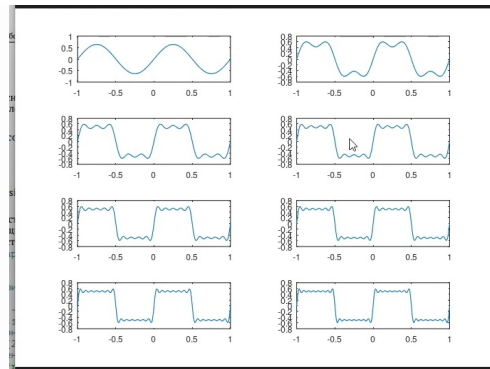
Далее скорректировали код для реализации меандра через синусы:

```
% meandr.m
% количество отсчетов (гармоник):
N=8;
% частота дискретизации:
t=-1:0.01:1;
% значение амплитуды:
A=1;
% период:
T=1;
% амплитуда гармоник
nh=(1:N)*2-1;
% массив коэффициентов для ряда, заданного через sin:
Am=2/pi ./ nh;
% Am(2:2:end) = -Am(2:2:end);
% массив гармоник:
harmonics=cos(2 * pi * nh * t/T);
% массив элементов ряда:
s1=harmonics.*repmat(Am',1,length(t));
% Суммирование ряда:
s2=cumsum(s1);
% Построение графиков:
for k=1:N
    subplot(4,2,k)
    plot(t, s2(k,:))
end
% Экспорт рисунка в файл .png:
print("meandr2.png");
```

Запустили сценарий и получили график меандра через синусы ([рис. @fig-010]), ([рис. @fig-011])

```
1 % meandr.m
2 % количество отсчетов (гармоник):
3 N=8;
4 % частота дискретизации:
5 t=-1:0.01:1;
6 % значение амплитуды:
7 A=1;
8 % период:
9 T=1;
10 % амплитуда гармоник
11 nh=(1:N)*2-1;
12 % массив коэффициентов для ряда, заданного ч
13 Am=2/pi ./ nh;
14 % массив гармоник:
15 harmonics=sin(2 * pi * nh * t/T);
16 % массив элементов ряда:
17 s1=harmonics.*repmat(Am,1,length(t));
18 % Суммирование ряда:
19 s2=cumsum(s1);
20 % Построение графиков:
21 for k=1:N
22     subplot(4,2,k)
23     plot(t, s2(k,:))
24 end
25
26 print('meandr2.png');
```

Работа программы meandr.m (2)



Графики меандра, содержащего различное число гармоник (через синусы)

Определение спектра и параметров сигнала

В нашем рабочем каталоге создали каталог *spectre1* и в нём новый сценарий с именем, *spectre.m*. Далее написали код для построения графика двух синусоидальных сигнала разной частоты:

```
% spectre1/spectre.m
% Создание каталогов signal и spectre для размещения графиков:
mkdir 'signal';
mkdir 'spectre';
% Длина сигнала (с):
tmax = 0.5;
% Частота дискретизации (Гц) (количество отсчётов):
fd = 512;
% Частота первого сигнала (Гц):
f1 = 10;
% Частота второго сигнала (Гц):
f2 = 40;
% Амплитуда первого сигнала:
a1 = 1;
% Амплитуда второго сигнала:
a2 = 0.7;
% Массив отсчётов времени:
t = 0:1./fd:tmax;
% Спектр сигнала:
fd2 = fd/2;
% Два сигнала разной частоты:
signal1 = a1*sin(2*pi*t*f1);
signal2 = a2*sin(2*pi*t*f2);
% График 1-го сигнала:
plot(signal1,'b');
% График 2-го сигнала:
hold on
plot(signal2,'r');
hold off
title('Signal')
% Экспорт графика в файл в каталоге signal:
print 'signal/spectre.png'
```

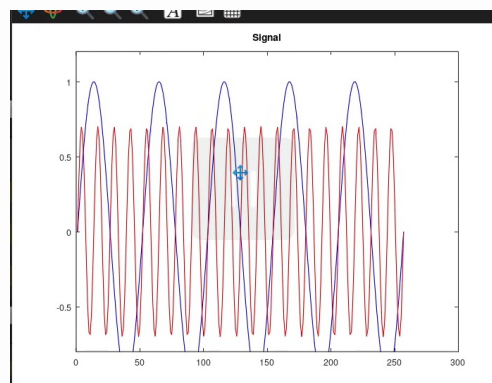
Зпустили сценарий и получили график сигналов ([рис. @fig-012]), ([рис. @fig-013])

```

1 % spectre1/spectre.m
2 % Создание каталогов signal и spectre для размещения
3 % файлов:
4 mkdir 'signal';
5 mkdir 'spectre';
6 % Частота сигнала (c):
7 tmax = 0.5;
8 % Частота дискретизации (Tn) (количество отсчетов):
9 fd = 512;
10 % Частота первого сигнала (Tn):
11 f1 = 10;
12 % Частота второго сигнала (Tn):
13 f2 = 40;
14 % Амплитуда первого сигнала:
15 a1 = 1;
16 % Амплитуда второго сигнала:
17 a2 = 0.7;
18 % Массив отсчетов времени:
19 t = 0:1/fd:tmax;
20 % Сумма сигнала:
21 fd2 = fd/2;
22 % Два сигнала разной частоты:
23 signal1 = a1*sin(2*pi*t*f1);
24 signal2 = a2*sin(2*pi*t*f2);
25 % График 1-го сигнала:
26 plot(signal1,'b');
27 % График 2-го сигнала:
28 hold on
29 plot(signal2,'r');
30 hold off
31 title('Signal');
32 % Сохранение графика в файл в каталоге signal:

```

Работа программы spectre.m (1)



Два синусоидальных сигнала разной частоты

Далее с помощью быстрого преобразования Фурье нашли спектры сигналов. Для этого добавили в файл *spectre.m* следующий код:

```

% Посчитаем спектр
% Амплитуды преобразования Фурье сигнала 1:
spectre1 = abs(fft(signal1,fd));
% Амплитуды преобразования Фурье сигнала 2:
spectre2 = abs(fft(signal2,fd));
% Построение графиков спектров сигналов:
plot(spectre1,'b');
hold on
plot(spectre2,'r');
hold off
title('Spectre');
print 'spectre/spectre.png';

```

Запустили сценарий и получили график спектров ([рис. @fig-014]), ([рис. @fig-015])

```

tmax = 0.5;
% Частота дискретизации (Гц) (количество от
fd = 512;
% Частота первого сигнала (Гц):
f1 = 10;
% Частота второго сигнала (Гц):
f2 = 40;
% Амплитуда первого сигнала:
a1 = 1;
% Амплитуда второго сигнала:
a2 = 0.7;
% Массив отсчётов времени:
t = 0:1./fd tmax;
% Спектр сигнала:
fd2 = fd/2;
% Два сигнала разной частоты:
signal1 = a1*sin(2*pi*t*f1);
signal2 = a2*sin(2*pi*t*f2);
% График 1-го сигнала:
plot(signal1, 'b');
% График 2-го сигнала:
hold on
plot(signal2, 'r');
hold off
title('Signal');
% Экспорт графика в файл в каталоге signal:
print 'signal/spectre.png';

% Посчитаем спектр
% Амплитуды преобразования Фурье сигнала 1:
spectre1 = abs(fft(signal1 fd));

```

Работа программы spectre.m (2)

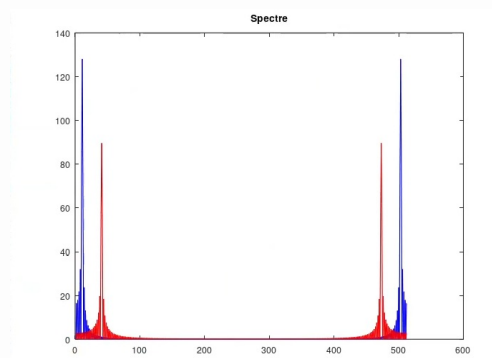


График спектров синусоидальных сигналов

Учитывая реализацию преобразования Фурье, скорректировали график спектра: отбросили дублирующие отрицательные частоты, а также приняли в расчёт то, что на каждом шаге вычисления быстрого преобразования Фурье происходит суммирование амплитуд сигналов. Для этого добавили в файл *spectre.m* следующий код:

```

% Исправление графика спектра
% Сетка частот:
f = 1000*(0:fd2)./(2*fd);
% Нормировка спектров по амплитуде:
spectre1 = 2*spectre1/fd2;
spectre2 = 2*spectre2/fd2;
% Построение графиков спектров сигналов:
plot(f,spectre1(1:fd2+1),'b');
hold on
plot(f,spectre2(1:fd2+1),'r');
hold off
xlim([0 100]);
title('Fixed spectre');
xlabel('Frequency (Hz)');
print 'spectre/spectre_fix.png';

```

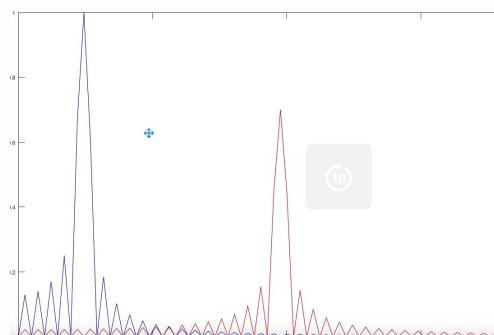
Запустили сценарий и получили исправленный графк спектров ([рис. @fig-016]), ([рис. @fig-017])


```

22 signal1 = a1*sin(2*pi*t*f1);
23 signal2 = a2*sin(2*pi*t*f2);
24 % График 1-го сигнала:
25 plot(signal1,'b');
26 % График 2-го сигнала:
27 hold on
28 plot(signal2,'r');
29 hold off
30 title('Signal');
31 % Экспорт графика в файл в каталоге signal:
32 print 'signal/signal.png';
33
34 % Посчитаем спектр
35 % Амплитуды преобразования Фурье сигнала 1:
36 spectre1 = abs(fft(signal1,fd));
37 % Амплитуды преобразования Фурье сигнала 2:
38 spectre2 = abs(fft(signal2,fd));
39 % Построение графиков спектров сигналов:
40 plot(spectre1,'b');
41 hold on
42 plot(spectre2,'r');
43 hold off
44 title('Spectrum');
45 print 'spectrum/spectrum.png';
46
47 % Исправление графика спектра
48 % Сетка частот:
49 f = 1000*(0:fd2)/(2*fd);
50 % Нормировка спектров по амплитуде:
51 spectre1 = 2*spectre1/fd2;
52 spectre2 = 2*spectre2/fd2;

```

Работа программы spectre.m (2)



Исправленный график спектров синусоидальных сигналов

Далее нашли спектр суммы рассмотренных сигналов, создав каталог *spectr_sum* и файл в нём *spectre_sum.m* со следующим кодом:

```

% spectr_sum/spectre_sum.m
% Создание каталогов signal и spectre для размещения графиков:
mkdir 'signal';
mkdir 'spectre';
% Длина сигнала (с):
tmax = 0.5;
% Частота дискретизации (Гц) (количество отсчётов):
fd = 512;
% Частота первого сигнала (Гц):
f1 = 10;
% Частота второго сигнала (Гц):
f2 = 40;
% Амплитуда первого сигнала:
a1 = 1;
% Амплитуда второго сигнала:
a2 = 0.7;
% Спектр сигнала
fd2 = fd/2;
% Сумма двух сигналов (синусоиды) разной частоты:
% Массив отсчётов времени:
t = 0:1./fd:tmax;
signal1 = a1*sin(2*pi*t*f1);
signal2 = a2*sin(2*pi*t*f2);
signal = signal1 + signal2;
plot(signal);
title('Signal');
print 'signal/spectre_sum.png';
% Подсчет спектра:
% Амплитуды преобразования Фурье сигнала:
spectre = fft(signal,fd);
% Сетка частот
f = 1000*(0:fd2)./(2*fd);
% Нормировка спектра по амплитуде:
spectre = 2*sqrt(spectre.*conj(spectre))./fd2;
% Построение графика спектра сигнала:
plot(f,spectre(1:fd2+1))
xlim([0 100]);
title('Spectre');
xlabel('Frequency (Hz)');
print 'spectre/spectre_sum.png';

```

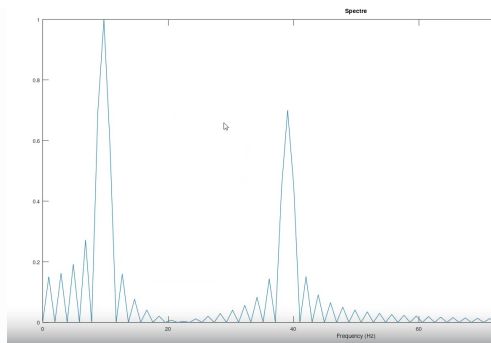
Запустили сценарий и получили нужные графики ([рис. @fig-018]), ([рис. @fig-019]), ([рис. @fig-020]), ([рис. @fig-021])

```

meandr.m x spectre.m x spectre_sum.m x
2 % Создание каталогов signal и spectre для р
3 % графиков:
4 mkdir 'signal';
5 mkdir 'spectre';
6 % Длина сигнала (с):
7 tmax = 0.5;
8 % Частота дискретизации (Гц) (количество от
9 fd = 512;
10 % Частота первого сигнала (Гц):
11 f1 = 10;
12 % Частота второго сигнала (Гц):
13 f2 = 40;
14 % Амплитуда первого сигнала:
15 a1 = 1;
16 % Амплитуда второго сигнала:
17 a2 = 0.7;
18 % Спектр сигнала
19 fd2 = fd/2;
20 % Сумма двух сигналов (синусоиды) разной ча
21 % Массив отсчётов времени:
22 t = 0:1./fd:tmax;
23 signal1 = a1*sin(2*pi*t*f1);
24 signal2 = a2*sin(2*pi*t*f2);
25 signal = signal1 + signal2;
26 plot(signal);
27 title('Signal');
28 print 'signal/spectre_sum.png';
29 % Подсчет спектра:
30 % Амплитуды преобразования Фурье сигнала:
31 spectre = fft(signal,fd);
32 % Сетка частот

```

Работа программы spectre_sum.m (1)



Суммарный сигнал

Амплитудная модуляция

В нашем рабочем каталоге создали каталог *modulation* и в нём новый сценарий с именем *am.m*. В нём прописали следующий код:

```
% modulation/am.m
% Создание каталогов signal и spectre для размещения графиков:
mkdir 'signal';
mkdir 'spectre';
% Модуляция синусоид с частотами 50 и 5
% Длина сигнала (с)
tmax = 0.5;
% Частота дискретизации (Гц) (количество отсчётов)
fd = 512;
% Частота сигнала (Гц)
f1 = 5;
% Частота несущей (Гц)
f2 = 50;
% Спектр сигнала
fd2 = fd/2;
% Построение графиков двух сигналов (синусоиды)
% разной частоты
% Массив отсчётов времени:
t = 0:1./fd:tmax;
signal1 = sin(2*pi*t*f1);
signal2 = sin(2*pi*t*f2);
signal = signal1 .* signal2;
plot(signal, 'b');
hold on
% Построение огибающей:
plot(signal1, 'r');
plot(-signal1, 'r');
hold off
title('Signal');
print 'signal/am.png';
% Расчет спектра:
% Амплитуды преобразования Фурье-сигнала:
spectre = fft(signal,fd);
% Сетка частот:
f = 1000*(0:fd2)./(2*fd);
% Нормировка спектра по амплитуде:
spectre = 2*sqrt(spectre.*conj(spectre))./fd2;
% Построение спектра:
plot(f,spectre(1:fd2+1), 'b')
xlim([0 100]);
title('Spectre');
xlabel('Frequency (Hz)');
print 'spectre/am.png';
```

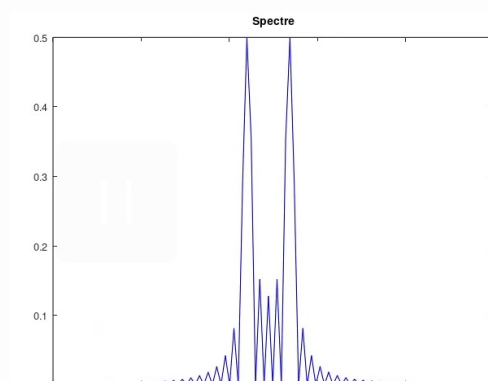
Запустили и получили нужные нам графики ([рис. @fig-022]), ([рис. @fig-023]), ([рис. @fig-024]), ([рис. @fig-025])

```

spectre.m  spectre_sum.m  am.m
% modulation/am.m
% Создание каталогов signal и spectre для р
% графиков:
mkdir 'signal';
mkdir 'spectre';
% Модуляция синусоид с частотами 50 и 5
% Длина сигнала (с)
tmax = 0.5;
% Частота дискретизации (Гц) (количество от
fd = 512;
% Частота сигнала (Гц)
f1 = 5;
% Частота несущей (Гц)
f2 = 50;
% Спектр сигнала
fd2 = fd/2;
% Построение графиков двух сигналов (синусо
% разной частоты
% Массив отсчетов времени:
t = 0:1./fd:tmax;
signal1 = sin(2*pi*t*f1);
signal2 = sin(2*pi*t*f2);
signal = signal1 .* signal2;
plot(signal, 'b');
hold on
% Построение огибающей:
plot(signal1, 'r');
plot(-signal1, 'r');
hold off
title('Signal');
print 'signal/fig-026';

```

Работа программы am.m (1)



Сигнал и огибающая при амплитудной модуляции

Кодирование сигнала. Исследование свойства самосинхронизации сигнала

В окне интерпретатора команд проверили, установлен ли у нас пакет расширений signal: >> pkg list ([рис. @fig-026])

Command Window			
fuzzy-logic-toolkit		0.6.2	...\mingw64\sh
ga		0.10.4	...\Octave-10.
general		2.1.3	...\Octave-10.
generate_html		0.3.3	...\mingw64\sh
geometry		4.1.0	...\Octave-10.
gsl		2.1.1	...\Octave-10.
image		2.16.1	...\Octave-10.
image-acquisition		0.3.3	...\mingw64\sh
instrument-control		0.9.5	...\mingw64\sh
interval		3.2.1	...\Octave-10.
io		2.7.0	...\Octave-10.
linear-algebra		2.2.3	...\mingw64\sh
lssa		0.1.4	...\Octave-10.
ltfat		2.6.0	...\Octave-10.
mapping		1.4.3	...\Octave-10.
matgeom		1.2.4	...\Octave-10.
miscellaneous		1.3.1	...\mingw64\sh
mqtt		0.0.5	...\Octave-10.
nan		3.7.0	...\Octave-10.
netcdf		1.0.18	...\Octave-10.
nurbs		1.4.4	...\Octave-10.
ocs		0.1.5	...\Octave-10.
octproj		3.0.2	...\Octave-10.
optim		1.6.2	...\Octave-10.
optiminterp		0.3.7	...\mingw64\sh
parallel		4.0.2	...\Octave-10.
quaternion		2.4.0	...\mingw64\sh
queueing		1.2.8	...\Octave-10.
signal		1.4.6	...\Octave-10.
sockets		1.4.1	...\Octave-10.
sparsersb		1.0.9	...\mingw64\sh
spline		1.3.5	...\Octave-10.
statistics		1.7.4	...\mingw64\sh

Пакет расширений signal

В нашем рабочем каталоге создали каталог *coding* и в нём файлы *main.m*, *maptowave.m*, *unipolar.m*, *ami.m*, *bipolarnrz.m*, *bipolarrrz.m*, *manchester.m*, *diffmanc.m*, *calcspectre.m* (рис. @fig-027])

octavelb > modulation > coding			Поиск в:
Имя			Дата изменения
main.m			13.09.2025 20:28
maptowave.m			13.09.2025 20:29
unipolar.m			13.09.2025 20:29
ami.m			13.09.2025 20:29
bipolarrrz.m			13.09.2025 20:29
manchester.m			13.09.2025 20:29
diffmanc.m			13.09.2025 20:30
calcspectre.m			13.09.2025 20:30

Структура каталога

В файле *main.m* подключили пакет *signal*, задали входные кодовые последовательности, прописали вызовы функций для построения графиков модуляций кодированных сигналов для кодовой последовательности *data*, прописали вызовы функций для построения графиков модуляций кодированных сигналов для кодовой последовательности *data_sync* и прописали вызовы функций для построения графиков спектров:

```
% coding/main.m
% Подключение пакета signal:
pkg load signal;
% Входная кодовая последовательность:
data=[0 1 0 0 1 1 0 0 0 1 1 0];
% Входная кодовая последовательность для проверки свойства
самосинхронизации:
data_sync=[0 0 0 0 0 0 0 1 1 1 1 1 1];
% Входная кодовая последовательность для построения спектра сигнала:
data_spectre=[0 1 0 1 0 1 0 1 0 1 0 1 0 1];
% Создание каталогов signal, sync и spectre для размещения графиков:
mkdir 'signal';
mkdir 'sync';
mkdir 'spectre';
```

```

axis("auto");

% Униполярное кодирование
wave=unipolar(data);
plot(wave);
ylim([-1 6]);
title('Unipolar');
print 'signal/unipolar.png';

% Кодирование ami
wave=ami(data);
plot(wave);
title('AMI');
print 'signal/ami.png';
% Кодирование NRZ
wave=bipolarnrz(data);
plot(wave);
title('Bipolar Non-Return to Zero');
print 'signal/bipolarnrz.png';
% Кодирование RZ
wave=bipolarrz(data);
plot(wave);
title('Bipolar Return to Zero');
print 'signal/bipolarrz.png';
% Манчестерское кодирование
wave=manchester(data);
plot(wave);
title('Manchester');
print 'signal/manchester.png';
% Дифференциальное манчестерское кодирование
wave=diffmanc(data);
plot(wave);
title('Differential Manchester');
print 'signal/diffmanc.png';

% Униполярное кодирование
wave=unipolar(data_sync);
plot(wave);
ylim([-1 6]);
title('Unipolar');
print 'sync/unipolar.png';
% Кодирование AMI
wave=ami(data_sync);
plot(wave);
title('AMI');
print 'sync/ami.png';
% Кодирование NRZ
wave=bipolarnrz(data_sync);
plot(wave);
title('Bipolar Non-Return to Zero');
print 'sync/bipolarnrz.png';
% Кодирование RZ
wave=bipolarrz(data_sync);
plot(wave);
title('Bipolar Return to Zero');
print 'sync/bipolarrz.png';
% Манчестерское кодирование
wave=manchester(data_sync);
plot(wave);
title('Manchester');
print 'sync/manchester.png';
% Дифференциальное манчестерское кодирование
wave=diffmanc(data_sync);
plot(wave);
title('Differential Manchester');
print 'sync/diffmanc.png';

% Униполярное кодирование:
wave=unipolar(data_spectre);
spectre=calcspectre(wave);
title('Unipolar');
print 'spectre/unipolar.png';
% Кодирование AMI:
wave=ami(data_spectre);

```

```

spectre=calcspectre(wave);
title('AMI');
print 'spectre/ami.png';
% Кодирование NRZ:
wave=bipolarnrz(data_spectre);
spectre=calcspectre(wave);
title('Bipolar Non-Return to Zero');
print 'spectre/bipolarnrz.png';
% Кодирование RZ:
wave=bipolarrz(data_spectre);
spectre=calcspectre(wave);
title('Bipolar Return to Zero');
print 'spectre/bipolarrz.png';
% Манчестерское кодирование:
wave=manchester(data_spectre);
spectre=calcspectre(wave);
title('Manchester');
print 'spectre/manchester.png';
% Дифференциальное манчестерское кодирование:
wave=diffmanc(data_spectre);
spectre=calcspectre(wave);
title('Differential Manchester');
print 'spectre/diffmanc.png';% coding/main.m
% Подключение пакета signal:
pkg load signal;
% Входная кодовая последовательность:
data=[0 1 0 0 1 1 0 0 1 1 0];
% Входная кодовая последовательность для проверки свойства
самосинхронизации:
data_sync=[0 0 0 0 0 0 1 1 1 1 1 1];
% Входная кодовая последовательность для построения спектра сигнала:
data_spectre=[0 1 0 1 0 1 0 1 0 1 0 1];
% Создание каталогов signal, sync и spectre для размещения графиков:
mkdir 'signal';
mkdir 'sync';
mkdir 'spectre';
axis("auto");

% Униполярное кодирование
wave=unipolar(data);
plot(wave);
ylim([-1 6]);
title('Unipolar');
print 'signal/unipolar.png';

% Кодирование ami
wave=ami(data);
plot(wave)
title('AMI');
print 'signal/ami.png';
% Кодирование NRZ
wave=bipolarnrz(data);
plot(wave);
title('Bipolar Non-Return to Zero');
print 'signal/bipolarnrz.png';
% Кодирование RZ
wave=bipolarrz(data);
plot(wave)
title('Bipolar Return to Zero');
print 'signal/bipolarrz.png';
% Манчестерское кодирование
wave=manchester(data);
plot(wave)
title('Manchester');
print 'signal/manchester.png';
% Дифференциальное манчестерское кодирование
wave=diffmanc(data);
plot(wave)
title('Differential Manchester');
print 'signal/diffmanc.png';

% Униполярное кодирование
wave=unipolar(data_sync);
plot(wave);

```

```

ylim([-1 6]);
title('Unipolar');
print 'sync/unipolar.png';
% Кодирование AMI
wave=ami(data_sync);
plot(wave)
title('AMI');
print 'sync/ami.png';
% Кодирование NRZ
wave=bipolarnrz(data_sync);
plot(wave);
title('Bipolar Non-Return to Zero');
print 'sync/bipolarnrz.png';
% Кодирование RZ
wave=bipolarrz(data_sync);
plot(wave)
title('Bipolar Return to Zero');
print 'sync/bipolarrz.png';
% Манчестерское кодирование
wave=manchester(data_sync);
plot(wave)
title('Manchester');
print 'sync/manchester.png';
% Дифференциальное манчестерское кодирование
wave=diffmanc(data_sync);
plot(wave)
title('Differential Manchester');
print 'sync/diffmanc.png';

% Униполярное кодирование:
wave=unipolar(data_spectre);
spectre=calcspectre(wave);
title('Unipolar');
print 'spectre/unipolar.png';
% Кодирование AMI:
wave=ami(data_spectre);
spectre=calcspectre(wave);
title('AMI');
print 'spectre/ami.png';
% Кодирование NRZ:
wave=bipolarnrz(data_spectre);
spectre=calcspectre(wave);
title('Bipolar Non-Return to Zero');
print 'spectre/bipolarnrz.png';
% Кодирование RZ:
wave=bipolarrz(data_spectre);
spectre=calcspectre(wave);
title('Bipolar Return to Zero');
print 'spectre/bipolarrz.png';
% Манчестерское кодирование:
wave=manchester(data_spectre);
spectre=calcspectre(wave);
title('Manchester');
print 'spectre/manchester.png';
% Дифференциальное манчестерское кодирование:
wave=diffmanc(data_spectre);
spectre=calcspectre(wave);
title('Differential Manchester');
print 'spectre/diffmanc.png';

```

([рис. @fig-028])


```

main.m
69 wave manchester(data_sync);
70 plot(wave);
71 title('Manchester');
72 print('wave/manchester.png');
73 % Дифференциальное манчестерское кодиро
74 wave diffmanc(data_sync);
75 plot(wave);
76 title('Differential Manchester');
77 print('wave/diffmanc.png');
78 % Униполярное кодирование:
79 wave unipolar(data_spectre);
80 spectre calcspectre(wave);
81 title('Unipolar');
82 print('spectre/unipolar.png');
83 % Кодирование AMI:
84 wave ami(data_spectre);
85 spectre calcspectre(wave);
86 title('AMI');
87 print('spectre/ami.png');
88 % Кодирование NRZ:
89 wave bipolarnrz(data_spectre);
90 spectre calcspectre(wave);
91 title('Bipolar Non-Return to Zero');
92 print('spectre/bipolarnrz.png');
93 % Кодирование RZ:
94 wave bipolarrrz(data_spectre);
95 spectre calcspectre(wave);
96 title('Bipolar Return to Zero');
97 print('spectre/bipolarrrz.png');
98 % Манчестерское кодирование:
99 wave manchester(data_spectre);

```

Файл main.m

Далее в файле *maptowave.m* прописали функцию, которая по входному битовому потоку строит график сигнала:

```

% coding/maptowave.m
function wave=maptowave(data)
    data=upsample(data,100);
    wave=filter(5*ones(1,100),1,data);

```

([рис. @fig-029])

```

main.m  maptowave.m
1 % coding/maptowave.m
2 function wave=maptowave(data)
3 data=upsample(data,100);
4 wave=filter(5*ones(1,100),1,data);
5

```

Файл maptowave.m

В файлах *unipolar.m*, *ami.m*, *bipolarnrz.m*, *bipolarrrz.m*, *manchester.m*, *diffmanc.m* прописали соответствующие функции преобразования кодовой:

Униполярное кодирование:

```

% coding/unipolar.m
% Униполярное кодирование:
function wave=unipolar(data)
    wave=maptowave(data);

```

([рис. @fig-030])

```

main.m  maptowave.m  unipolar.m
1 % coding/unipolar.m
2 % Униполярное кодирование:
3 function wave=unipolar(data)
4     wave=maptowave(data);

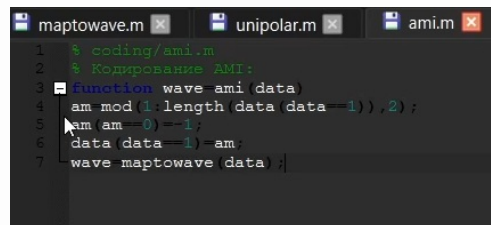
```

Файл unipolar.m, Униполярное кодирование

Кодирование AMI:

```
% coding/ami.m
% Кодирование AMI:
function wave=ami(data)
    am=mod(1:length(data(data==1)),2);
    am(am==0)=-1;
    data(data==1)=am;
    wave=maptowave(data);
```

([рис. @fig-031])



Файл ami.m, Кодирование AMI

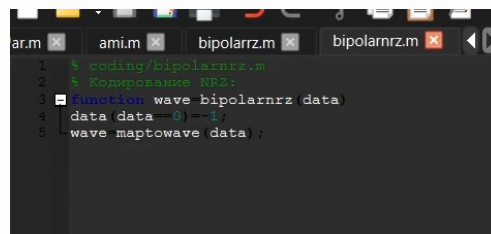
Кодирование NRZ:

```
% coding/bipolarnrz.m
% Кодирование NRZ:
function wave=bipolarnrz(data)
    data(data==0)=-1;
    wave=maptowave(data);
```

Кодирование RZ:

```
% coding/bipolarrz.m
% Кодирование RZ:
function wave=bipolarrz(data)
    data(data==0)=-1;
    data=upsample(data,2);
    wave=maptowave(data);
```

([рис. @fig-033])



Файл bipolarrrz.m, Кодирование RZ

Манчестерское кодирование:

```
% coding/manchester.m
% Манчестерское кодирование:
function wave=manchester(data)
    data(data==0)=-1;
    data=upsample(data,2);
    data=filter([-1 1],1,data);
    wave=maptowave(data);
```

([рис. @fig-034])

```

1 % coding/diffmanc.m
2 % Дифференциальное манчестерское кодирование
3 function wave=diffmanc(data)
4 data=filter(1,[1 1],data);
5 data=mod(data,2);
6 wave=manchester(data);

```

Файл manchester.m, Манчестерское кодирование

Дифференциальное манчестерское кодирование:

```

% coding/diffmanc.m
% Дифференциальное манчестерское кодирование
function wave=diffmanc(data)
    data=filter(1,[1 1],data);
    data=mod(data,2);
    wave=manchester(data);

```

([рис. @fig-035])

```

1 % coding/manchester.m
2 % Манчестерское кодирование:
3 function wave=manchester(data)
4 data(data==0)=-1;
5 data=upsample(data,2);
6 data=filter([-1 1],1,data);
7 wave=maptowave(data);
8

```

Файл diffmanc.m, Дифференциальное манчестерское кодирование

В файле calcspectre.m прописали функцию построения спектра сигнала:

```

% calcspectre.m
% Функция построения спектра сигнала:
function spectre = calcspectre(wave)
    % Частота дискретизации (Гц):
    Fd = 512;
    Fd2 = Fd/2;
    Fd3 = Fd/2 + 1;
    X = fft(wave,Fd);
    spectre = X.*conj(X)/Fd;
    f = 1000*(0:Fd2)/Fd;
    plot(f,spectre(1:Fd3));
    xlabel('Frequency (Hz)');

```

([рис. @fig-036])

```

1 % calcspectre.m
2 % функция построения спектра сигнала:
3 function spectre = calcspectre(wave)
4 % Частота дискретизации (Гц):
5 Fd = 512;
6 Fd2 = Fd/2;
7 Fd3 = Fd/2 + 1;
8 X = fft(wave,Fd);
9 spectre = X.*conj(X)/Fd;
10 f = 1000*(0:Fd2)/Fd;
11 plot(f,spectre(1:Fd3));
12 xlabel('Frequency (Hz)');
13

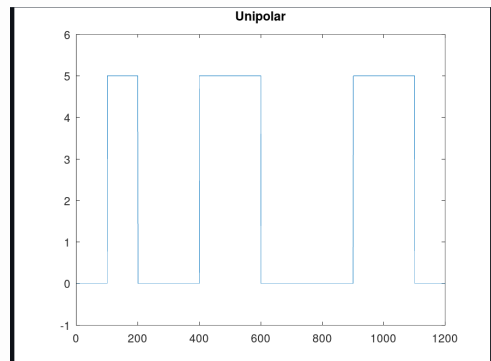
```

Файл calcspectre.m

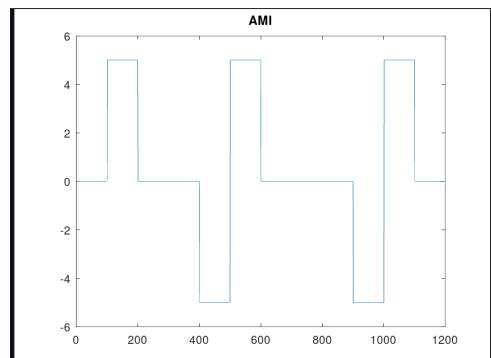
Запустили главный скрипт main.m. В каталоге *signal* бфли получены файлы с

графиками кодированного сигнала (рис. 3.40-3.45), в каталоге *sync* — файлы с графиками, иллюстрирующими свойства самосинхронизации (рис. 3.46–3.51), в каталоге *spectre* — файлы с графиками спектров сигналов (рис. 3.52–3.57) ([рис. @fig-037]), ([рис. @fig-038]), ([рис. @fig-039])

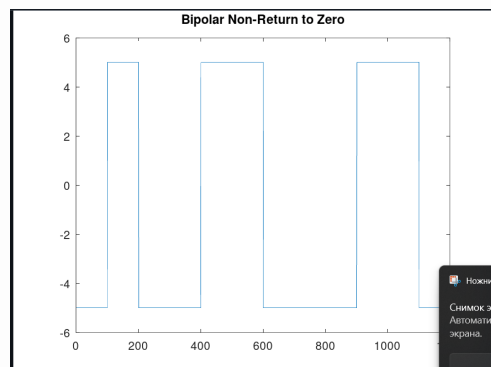
Полученные графики ([рис. @fig-040]), ([рис. @fig-041]), ([рис. @fig-042]), ([рис. @fig-043]), ([рис. @fig-044]), ([рис. @fig-045]), ([рис. @fig-046]), ([рис. @fig-047]), ([рис. @fig-048]), ([рис. @fig-049]), ([рис. @fig-050]), ([рис. @fig-051]), ([рис. @fig-052]), ([рис. @fig-053]), ([рис. @fig-054]), ([рис. @fig-055]), ([рис. @fig-056]), ([рис. @fig-057])



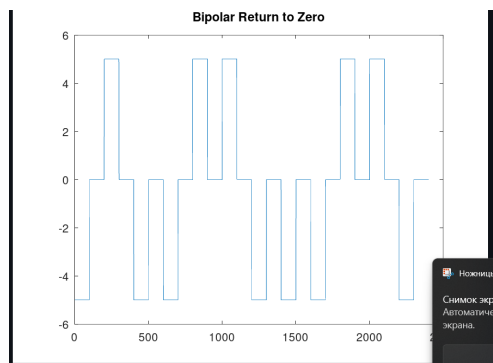
Униполярное кодирование



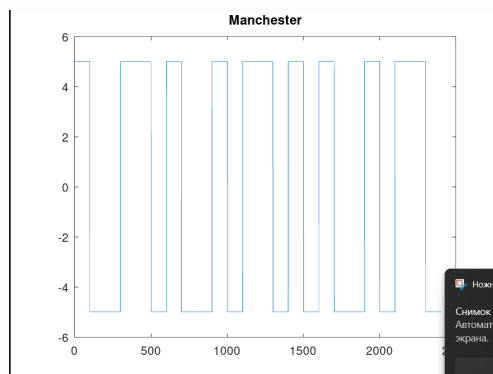
Кодирование AMI



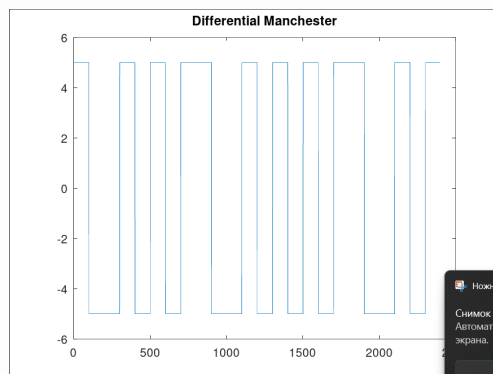
Кодирование NRZ



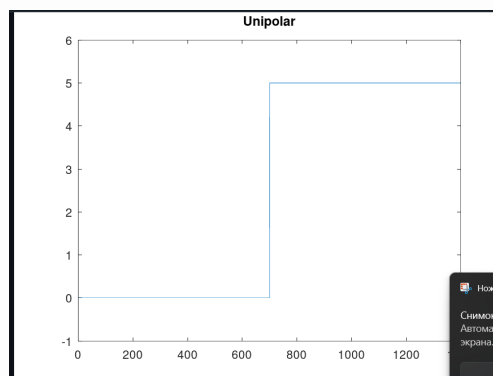
Кодирование RZ



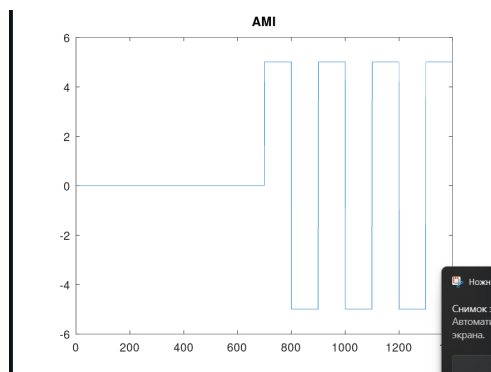
Манчестерское кодирование



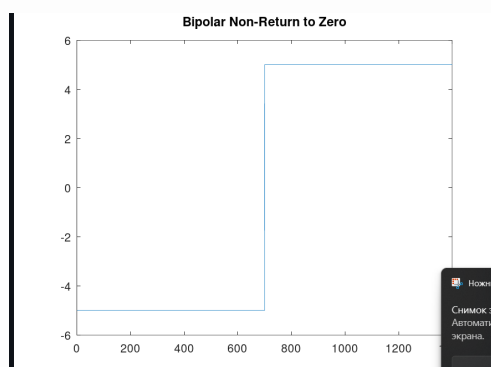
Дифференциальное манчестерское кодирование



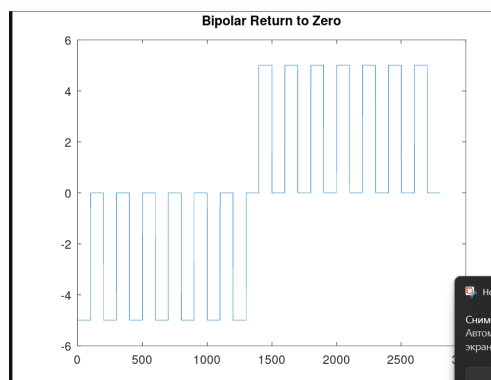
Униполярное кодирование: нет самосинхронизации



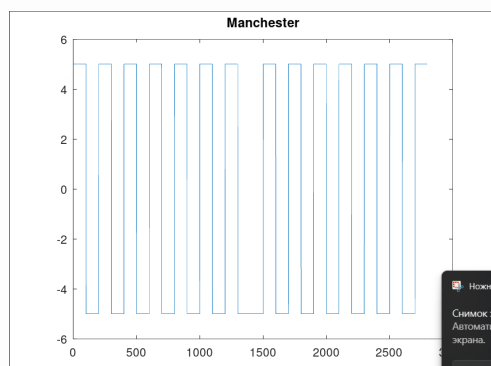
Кодирование AMI: самосинхронизация при наличии сигнала



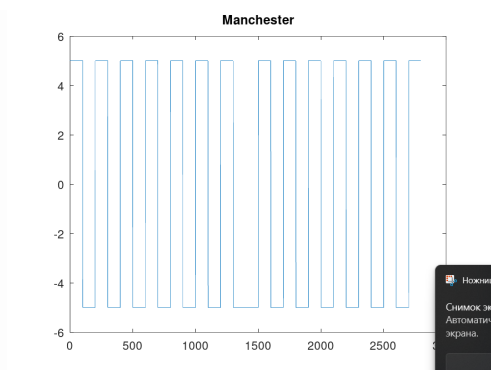
Кодирование NRZ: нет самосинхронизации



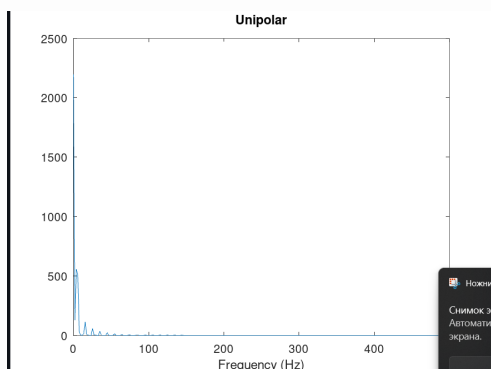
Кодирование RZ: есть самосинхронизация



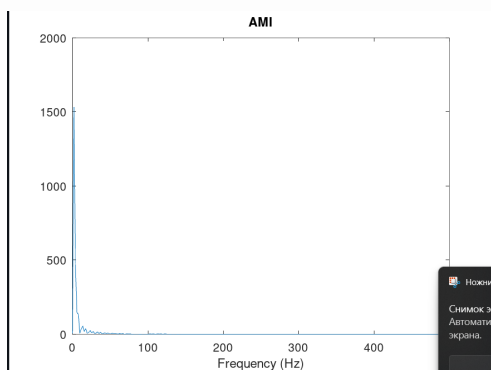
Манчестерское кодирование: есть самосинхронизация



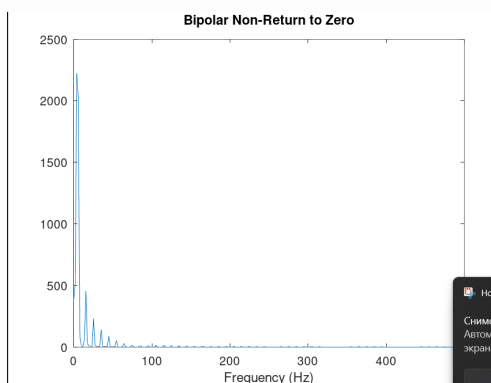
Дифференциальное манчестерское кодирование: есть самосинхронизация



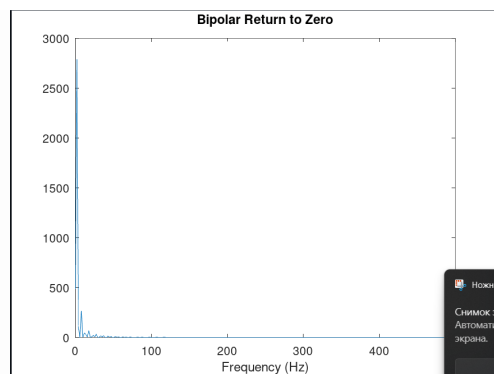
Униполярное кодирование: спектр сигнала



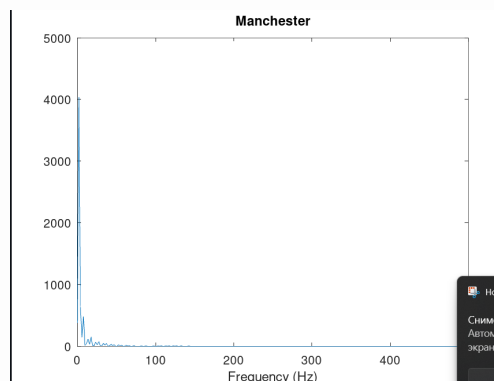
Кодирование AMI: спектр сигнала



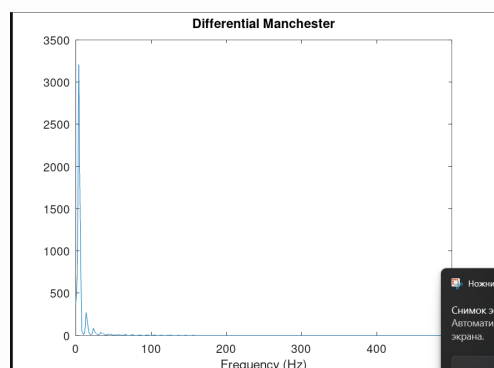
Кодирование NRZ: спектр сигнала



Кодирование RZ: спектр сигнала



Манчестерское кодирование: спектр сигнала



Дифференциальное манчестерское кодирование: спектр сигнала

Выводы

В ходе выполнения лабораторной работы №1 мы изучили методы кодирования и модуляции сигналов с помощью высокоуровневого языка программирования Octave. Определили спектр и параметры сигнала. Продемонстрировали принципы модуляции сигнала на примере аналоговой амплитудной модуляции. Исследовали свойства самосинхронизации сигнала.

Список литературы

1. Лабораторная работа №1